LAB REPORT

Fundamentals of Operating Systems (OS)

Assignment – 4

Name : Dhruv Gupta

Roll No : 2401201015

Course : BCA (AI & DS) [H]

Section : A

---

1. Introduction

Disk scheduling is a crucial function of an operating system that determines the order in which disk I/O requests are serviced. Since disk access involves mechanical movement of the read/write head, efficient scheduling is necessary to reduce seek time and improve system performance.

Different disk scheduling algorithms aim to optimize head movement, minimize seek time, and increase throughput. In this assignment, several important disk scheduling algorithms have been implemented and analyzed, including:

- First Come First Serve (FCFS)
- Shortest Seek Time First (SSTF)
- SCAN (Elevator Algorithm)
- C-SCAN (Circular SCAN)

The performance of each algorithm is evaluated based on total seek time and head movement.

2. Objectives

- To understand disk scheduling concepts
- To implement various disk scheduling algorithms
- To calculate total head movement (seek time)
- To compare the performance of different algorithms
- To analyze the efficiency of each algorithm

3. Tools and Technologies Used

- Programming Language: Python 3.x
- Code Editor: Text File
- Platform: Ubuntu (Linux)
- Libraries Used: Basic Python libraries

4. Input and Disk Request Representation

4.1 Description

The program takes the following inputs:

- Number of disk requests
- Disk request sequence
- Initial head position
- Disk size

These inputs simulate real-world disk request handling.

4.2 Example Input

- Requests: 98 183 37 122 14 124 65 67
- Initial Head Position: 53
- Disk Size: 200

```
(venv) dhruvgupta@dhruvgupta:~/Downloads$ python ass4.py
Enter number of disk requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Request Queue: [98, 183, 37, 122, 14, 124, 65, 67]
Initial Head Position: 53
```

5. FCFS Disk Scheduling Algorithm

5.1 Description

The First Come First Serve (FCFS) algorithm services disk requests in the order they arrive.

5.2 Working

- Requests are processed sequentially
- The head moves from one request to the next
- Total seek time is calculated as the sum of movements

5.3 Characteristics

- Simple and easy to implement
- Can lead to large seek times due to random access

```
--- FCFS ---
Move from 53 to 98 (Seek = 45)
Move from 98 to 183 (Seek = 85)
Move from 183 to 37 (Seek = 146)
Move from 37 to 122 (Seek = 85)
Move from 122 to 14 (Seek = 108)
Move from 14 to 124 (Seek = 110)
Move from 124 to 65 (Seek = 59)
Move from 65 to 67 (Seek = 2)
Sequence: [98, 183, 37, 122, 14, 124, 65, 67]
Total Seek Time: 640
```

6. SSTF Disk Scheduling Algorithm

6.1 Description

Shortest Seek Time First (SSTF) selects the request closest to the current head position.

6.2 Working

- Find the nearest request
- Move head to that request
- Repeat until all requests are served

6.3 Characteristics

- Reduces seek time compared to FCFS
- May cause starvation of distant requests

```
--- SSTF ---
Move from 53 to 65 (Seek = 12)
Move from 65 to 67 (Seek = 2)
Move from 67 to 37 (Seek = 30)
Move from 37 to 14 (Seek = 23)
Move from 14 to 98 (Seek = 84)
Move from 98 to 122 (Seek = 24)
Move from 122 to 124 (Seek = 2)
Move from 124 to 183 (Seek = 59)
Sequence: [65, 67, 37, 14, 98, 122, 124, 183]
Total Seek Time: 236
```

7. SCAN (Elevator Algorithm)

7.1 Description

SCAN moves the disk head in one direction servicing requests, then reverses direction when it reaches the end.

7.2 Working

- Divide requests into left and right of head
- Move in one direction servicing requests
- Reverse direction at disk end

7.3 Characteristics

- Provides better performance than FCFS
- Balanced head movement

```
--- SCAN ---
Move from 53 to 65 (Seek = 12)
Move from 65 to 67 (Seek = 2)
Move from 67 to 98 (Seek = 31)
Move from 98 to 122 (Seek = 24)
Move from 122 to 124 (Seek = 2)
Move from 124 to 183 (Seek = 59)
Move from 183 to 199 (Seek = 16)
Move from 199 to 37 (Seek = 162)
Move from 37 to 14 (Seek = 23)
Sequence: [65, 67, 98, 122, 124, 183, 37, 14]
Total Seek Time: 331
```

8. C-SCAN (Circular SCAN)

8.1 Description

C-SCAN moves the head in one direction only. After reaching the end, it jumps back to the beginning and continues.

8.2 Working

- Move in one direction servicing requests
- Jump to beginning after reaching end
- Continue servicing remaining requests

8.3 Characteristics

- Provides uniform wait time
- More fair compared to SCAN

```
--- C-SCAN ---
Move from 53 to 65 (Seek = 12)
Move from 65 to 67 (Seek = 2)
Move from 67 to 98 (Seek = 31)
Move from 98 to 122 (Seek = 24)
Move from 122 to 124 (Seek = 2)
Move from 124 to 183 (Seek = 59)
Move from 183 to 199 (Seek = 16)
Jump from 199 to 0
Move from 0 to 14 (Seek = 14)
Move from 14 to 37 (Seek = 23)
Sequence: [65, 67, 98, 122, 124, 183, 14, 37]
Total Seek Time: 382
```

9. Performance Comparison

9.1 Description

The total seek time for each algorithm is calculated and compared.

9.2 Output

The results display:

- FCFS seek time
- SSTF seek time
- SCAN seek time
- C-SCAN seek time

```
==================================
PERFORMANCE COMPARISON
==================================
FCFS Seek Time: 640
SSTF Seek Time: 236
SCAN Seek Time: 331
C-SCAN Seek Time: 382
```

10. Result Analysis

10.1 Observations

- SSTF generally produces lower seek time than FCFS
- SCAN provides balanced performance
- C-SCAN ensures uniform waiting time
- FCFS often results in higher seek time

10.2 Best and Worst Algorithm

- Best Algorithm: SSTF or SCAN (depends on input)
- Worst Algorithm: FCFS (usually highest seek time)

```
==================================
ANALYSIS
==================================
Best Algorithm: SSTF (Minimum Seek Time)
Worst Algorithm: FCFS (Maximum Seek Time)
```

11. Challenges Faced

- Implementing correct SSTF logic
- Managing direction change in SCAN
- Handling jump logic in C-SCAN
- Ensuring accurate seek time calculation

12. Learning Outcomes

- Understood disk scheduling techniques
- Learned head movement optimization
- Compared algorithm performance
- Improved logical and coding skills

13. Conclusion

Disk scheduling algorithms significantly impact system performance. This assignment demonstrates how different algorithms affect seek time and efficiency. While FCFS is simple, algorithms like SSTF, SCAN, and C-SCAN provide better optimization and performance in real-world systems.

14. GitHub Repository Link

👉 Github Link